

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KINH TẾ - LUẬT

KHOA TOÁN KINH TẾ



ĐỒ ÁN CUỐI KỲ

ỨNG DỤNG HỌC MÁY TRONG VIỆC DỰ
ĐOÁN MỨC ĐỘ HÀI LÒNG CỦA KHÁCH DU
LỊCH DỰA TRÊN DỮ LIỆU ĐÁNH GIÁ TỪ
NỀN TẢNG BOOKING.COM

Giảng viên: **Th.S Trương Quang Nhật**
Sinh viên thực hiện: **Nguyễn Thiên Quốc**
MSSV: **K234131503**
Mã học phần: **252BMA4006**

Tp. HCM, ngày 29 tháng 03 năm 2026

LỜI CẢM ƠN

Em xin gửi lời cảm ơn chân thành đến Thầy Trương Quang Nhật vì đã tận tâm giảng dạy và truyền đạt cho em những kiến thức nền tảng quan trọng trong Machine Learning. Những bài giảng rõ ràng, mạch lạc cùng với cách tiếp cận logic của thầy đã giúp em hiểu sâu hơn về các nguyên lý tối ưu hóa trong học máy. Nhờ sự hướng dẫn tận tình và những kiến thức quý báu mà thầy đã chia sẻ, em có thể vận dụng hiệu quả các mô hình học máy trong quá trình thực hiện đồ cuối kỳ của mình. Sự hỗ trợ và động viên của thầy là nguồn động lực lớn giúp em hoàn thành đồ án một cách nghiêm túc và tốt nhất. Em xin chân thành cảm ơn thầy vì sự tận tâm và đồng hành trong suốt quá trình học tập và nghiên cứu vừa qua.

LỜI CAM KẾT

Tôi xin cam kết rằng toàn bộ nội dung trong đồ án này là kết quả của quá trình nghiên cứu, tìm hiểu và thực hiện của chính mình. Các tài liệu tham khảo, số liệu và nguồn trình dẫn đều được nêu rõ và tuân thủ quy định. Tôi hoàn toàn chịu trách nhiệm trước nhà trường và pháp luật về tính trung thực và chính xác của nội dung trong đồ án.

MỤC LỤC

LỜI CẢM ƠN.....	I
LỜI CAM KẾT.....	II
DANH MỤC BẢNG BIỂU.....	V
DANH MỤC HÌNH ẢNH.....	VI
LỜI MỞ ĐẦU	1
CHƯƠNG 1: GIỚI THIỆU DỮ LIỆU	2
1.1. Mô tả bộ dữ liệu	2
1.2. Các biến dữ liệu chính và ý nghĩa thực tiễn	3
1.3. Kiểu dữ liệu và thống kê mô tả	4
1.3.1. Kiểu dữ liệu và tình trạng dữ liệu	4
1.3.2. Thống kê mô tả dữ liệu	5
CHƯƠNG 2: GIỚI THIỆU BÀI TOÁN	8
2.1. Phát biểu bài toán	8
2.2. Loại bài toán, biến đầu vào và biến mục tiêu.....	8
2.2.1. Phân loại nhị phân trong học máy.....	8
2.2.2. Biến đầu vào và biến mục tiêu.....	9
2.3. Mô hình sử dụng và quy trình thực hiện	11
2.3.1. Mô hình sử dụng	11
2.3.2. Quy trình thực hiện	13
CHƯƠNG 3: XỬ LÝ DỮ LIỆU	14
3.1. Làm sạch dữ liệu và xử lý dữ liệu bị thiếu	14
3.2. Mã hóa biến phân loại và chuẩn hóa biến số.....	17
3.3. Phân tích tương quan và chia tập dữ liệu	18
CHƯƠNG 4: MÔ HÌNH HÓA DỮ LIỆU.....	21
4.1. Mô hình Logistic Regression	21
4.1.1. Cơ sở lý thuyết	21
4.1.2. Áp dụng mô hình.....	22
4.2. Mô hình Support Vector Machine	23

4.2.1. Cơ sở lý thuyết	23
4.2.2. Áp dụng mô hình.....	25
CHƯƠNG 5: ĐÁNH GIÁ MÔ HÌNH.....	27
5.1. Các tiêu chí đánh giá	27
5.1.1. Ma trận nhầm lẫn	27
5.1.2. Các chỉ số đánh giá	28
5.2. Kết quả và so sánh các mô hình	29
5.2.1. Kết quả từ Logistic Regression.....	29
5.2.2. Kết quả từ SVM	31
5.2.3. So sánh kết quả hai mô hình	32
5.3. Kết quả dự đoán của mô hình.....	35
5.4. Minh họa dự đoán mức độ hài lòng của khách du lịch	35
CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	37
6.1. Kết luận chung.....	37
6.2. Hạn chế của đề tài.....	38
TÀI LIỆU THAM KHẢO	39

DANH MỤC BẢNG BIỂU

Bảng 1: Các biến của bộ dữ liệu.....	4
--------------------------------------	---

DANH MỤC HÌNH ẢNH

Hình 1.1. Khái quát về bộ dữ liệu.....	2
Hình 1.2. Kiểm tra kích thước dữ liệu.....	2
Hình 1.3. Kiểu dữ liệu của các biến	4
Hình 1.4. Thống kê mô tả cho dữ liệu	5
Hình 1.5. Phân bố điểm đánh giá khách sạn (Rating)	6
Hình 1.6. Phân bố giá phòng (Room Price).....	7
Hình 2.1. Kết quả thực hiện chuẩn hóa và tạo biến mục tiêu.....	9
Hình 2.2. Biểu đồ phân bố biến mục tiêu trong tập dữ liệu	10
Hình 3.1. Kiểm tra dữ liệu bị thiếu.....	14
Hình 3.2. Kiểm tra dữ liệu sau khi xử lý	16
Hình 3.3. Ma trận tương quan giữa các biến trong tập dữ liệu.....	18
Hình 4.1. Pipeline train dữ liệu bằng mô hình Logistic Regression	22
Hình 4.2. Minh họa siêu phẳng tối ưu và khoảng biên trong SVM	24
Hình 5.1. Biểu đồ radar so sánh hiệu suất của hai mô hình phân loại.....	32
Hình 5.2. Ma trận nhầm lẫn của Logistic Regression	33
Hình 5.3. Ma trận nhầm lẫn của SVM.....	34
Hình 5.4. Kết quả dự đoán của mô hình.....	35
Hình 5.5. Kết quả cho bài toán giả định	36

LỜI MỞ ĐẦU

Trong những năm gần đây, với sự phát triển mạnh mẽ của Internet và các nền tảng thương mại điện tử, ngành du lịch và dịch vụ lưu trú đã có nhiều thay đổi trong cách khách hàng tìm kiếm và lựa chọn dịch vụ. Các nền tảng đặt phòng trực tuyến như Booking.com, TripAdvisor hay Expedia cho phép khách du lịch dễ dàng tìm kiếm thông tin, so sánh giá cả và tham khảo các đánh giá từ những khách hàng trước đó trước khi đưa ra quyết định đặt phòng. Theo thống kê, khoảng **95%** khách du lịch đọc các đánh giá trực tuyến trước khi đặt phòng và **93%** cho biết các đánh giá này ảnh hưởng trực tiếp đến quyết định của họ.

Bên cạnh đó, lượng dữ liệu đánh giá được tạo ra trên các nền tảng du lịch là vô cùng lớn. Chẳng hạn, TripAdvisor ghi nhận hơn **31 triệu** đánh giá của khách du lịch trong năm 2024, trong khi Booking.com cũng sở hữu hàng triệu đánh giá khách sạn được tạo ra liên tục từ người dùng trên toàn thế giới. Một nghiên cứu phân tích dữ liệu trên các ứng dụng du lịch cho thấy **81.9%** đánh giá trên Booking.com là tích cực, **11.3%** trung lập và khoảng **8.4%** tiêu cực. Điều này cho thấy các đánh giá trực tuyến không chỉ phản ánh cảm nhận của khách hàng mà còn cung cấp nguồn dữ liệu quan trọng để phân tích mức độ hài lòng đối với dịch vụ lưu trú.

Trong bối cảnh đó, việc khai thác và phân tích dữ liệu đánh giá bằng các phương pháp khoa học dữ liệu và học máy đang trở thành một hướng nghiên cứu quan trọng. Thông qua việc xây dựng các mô hình dự đoán, doanh nghiệp có thể hiểu rõ hơn mức độ hài lòng của khách du lịch và xác định các yếu tố ảnh hưởng đến trải nghiệm của khách hàng.

Xuất phát từ những lý do trên, đề tài **“Ứng dụng học máy trong việc dự đoán mức độ hài lòng của khách du lịch dựa trên dữ liệu đánh giá từ nền tảng Booking.com”** được thực hiện nhằm xây dựng mô hình dự đoán mức độ hài lòng của khách hàng dựa trên dữ liệu đánh giá khách sạn. Nghiên cứu áp dụng hai thuật toán Logistic Regression và Support Vector Machine để phân loại và so sánh hiệu quả dự đoán, qua đó góp phần hỗ trợ việc phân tích dữ liệu đánh giá và cải thiện chất lượng dịch vụ trong lĩnh vực du lịch và lưu trú.

CHƯƠNG 1: GIỚI THIỆU DỮ LIỆU

1.1. Mô tả bộ dữ liệu

Bộ dữ liệu sử dụng là “**Hotel Dataset: Rates, Reviews and Amenities**”, được lấy từ nền tảng chia sẻ dữ liệu Kaggle:

<https://www.kaggle.com/datasets/joyshil0599/hotel-dataset-rates-reviews-and-amenities5k/data>. Dữ liệu được thu thập nhằm phục vụ cho việc phân tích dữ liệu, nghiên cứu hành vi khách hàng cũng như ứng dụng trong các bài toán học máy trong lĩnh vực du lịch và khách sạn.

Dữ liệu được lưu trữ dưới dạng bảng, trong đó mỗi dòng đại diện cho một bản ghi dữ liệu và mỗi cột tương ứng với một thuộc tính cụ thể. Các thuộc tính trong bộ dữ liệu bao gồm thông tin về khách sạn, các tiện nghi, giá phòng,...và có ảnh hưởng lớn đến sự đánh giá và mức độ hài lòng của khách hàng.

	Hotel Name	Location	Rating	Review Score	Number of	Room Score	Room Type	Bed Type	Room Price (in BDT or any other currency)
0	Krabi La Playa Resort - SHA Plus	Ao Nang Beach	8.2	Very Good	141	8.6	Deluxe Double or Twin Room	1 double or 2 twins	146026
1	KC Beach Club & Pool Villas	Chaweng City Center , Chaweng	7.1	Good	193	NaN	Double Room with Balcony and Sea View	1 queen bed	215304
2	Rawai VIP Villas & Kids Park	Rawai Beach	8.3	Very Good	63	8.9	2 Bedroom Pool Villa	2 queen beds	435384
3	FuramaXclusive Sathorn, Bangkok	Bang Rak, Bangkok	7.7	Good	1,304	8.1	Executive Double Room	1 full bed	146240
4	Bo Phut Resort and Spa - SHA Plus	Bophut	9.3	Wonderful	294	9.6	Villa with Garden View	1 double or 2 twins	621072

Hình 1.1. Khái quát về bộ dữ liệu

Sau khi tải và kiểm tra bộ dữ liệu bằng thư viện Pandas trong Python, kích thước của bộ dữ liệu được xác định bằng lệnh `df.shape`. Kết quả cho thấy bộ dữ liệu có kích thước (3465, 9), nghĩa là dữ liệu bao gồm 3465 dòng và 9 cột.

```
# Kích thước dữ liệu
df.shape
```

```
(3465, 9)
```

Hình 1.2. Kiểm tra kích thước dữ liệu

Trong đó, 3465 dòng tương ứng với 3465 bản ghi dữ liệu, mỗi bản ghi đại diện cho thông tin của một khách sạn hoặc một đánh giá liên quan đến khách sạn. 9 cột tương ứng với 9 biến (thuộc tính) mô tả các đặc điểm khác nhau.

1.2. Các biến dữ liệu chính và ý nghĩa thực tiễn

Bộ dữ liệu **booking_hotel** bao gồm 9 biến với các ý nghĩa thực tiễn khác nhau:

Tên biến	Mô tả	Ý nghĩa thực tiễn
Hotel Name	Tên của khách sạn	Giúp xác định và phân biệt các khách sạn trong bộ dữ liệu
Location	Vị trí của khách sạn	Cho biết khách sạn nằm ở khu vực hoặc thành phố nào, có thể ảnh hưởng đến giá phòng và mức độ hài lòng của khách
Rating	Điểm đánh giá tổng thể của khách sạn (thang 1 - 10)	Thể hiện mức độ đánh giá chung của khách hàng đối với khách sạn
Review Score	Điểm đánh giá dựa trên các nhận xét của khách hàng (thang 1 - 10)	Phản ánh mức độ hài lòng của khách du lịch sau khi sử dụng dịch vụ
Number of review	Tổng số lượng đánh giá của khách hàng	Cho biết mức độ phổ biến và mức độ quan tâm của khách đối với khách sạn
Room Score	Điểm đánh giá chất lượng phòng (thang 1 - 10)	Thể hiện mức độ hài lòng của khách hàng đối với chất lượng phòng
Room Type	Loại phòng (single, double, suite,...)	Cho biết các loại phòng mà khách sạn cung cấp
Bed Type	Loại giường trong phòng (king, queen, twin,...)	Phản ánh đặc điểm tiện nghi của phòng

Room Price	Giá phòng mỗi đêm	Thể hiện chi phí lưu trú và là một yếu tố ảnh hưởng đến quyết định lựa chọn khách sạn
-------------------	-------------------	---

Bảng 1: Các biến của bộ dữ liệu

1.3. Kiểu dữ liệu và thống kê mô tả

1.3.1. Kiểu dữ liệu và tình trạng dữ liệu

```
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3465 entries, 0 to 3464
Data columns (total 9 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   Hotel Name                               3465 non-null   object
1   Location                                 3465 non-null   object
2   Rating                                   3359 non-null   float64
3   Review Score                             3359 non-null   object
4   Number of                               3328 non-null   float64
5   Room Score                               2709 non-null   float64
6   Room Type                               3465 non-null   object
7   Bed Type                                3449 non-null   object
8   Room Price (in BDT or any other currency) 3465 non-null   int64
dtypes: float64(3), int64(1), object(5)
memory usage: 243.8+ KB
```

Hình 1.3. Kiểu dữ liệu của các biến

Để kiểm tra cấu trúc tổng thể của bộ dữ liệu, sử dụng hàm `df.info()` trong thư viện Pandas nhằm kiểm tra kiểu dữ liệu, số lượng quan sát và tình trạng thiếu dữ liệu của từng biến. Kết quả cho thấy tập dữ liệu bao gồm cả biến định tính và định lượng.

- Các biến có kiểu định tính (Object): Hotel Name, Location, Review Score, Room Type và Bed Type.

- Các biến có kiểu định lượng (float, int): Rating, Number of Reviews, Room Score và Room Price.

Từ kết quả `df.info()` có thể thấy một số biến có giá trị bị thiếu:

- Rating: thiếu 100 giá trị
- Review Score: thiếu 106 giá trị
- Number of reviews: thiếu 106 giá trị
- Room Score: thiếu 756 giá trị
- Bed Type: thiếu 16 giá trị

Trong đó, biến Room Score có số lượng giá trị thiếu nhiều nhất. Vì vậy, trong bước tiền xử lý dữ liệu, cần áp dụng các phương pháp như loại bỏ bản ghi thiếu hoặc điền giá trị để đảm bảo chất lượng dữ liệu trước khi phân tích.

1.3.2. Thống kê mô tả dữ liệu

	Rating	Number of	Room Score	Room Price (in BDT or any other currency)
count	3359.000000	3328.000000	2709.000000	3.465000e+03
mean	8.205389	1046.850361	8.779734	3.437072e+05
std	0.808736	1362.785639	0.472312	7.488245e+05
min	1.000000	2.000000	8.000000	2.588000e+03
25%	7.900000	183.000000	8.400000	1.504710e+05
50%	8.300000	556.000000	8.800000	2.266210e+05
75%	8.700000	1422.000000	9.100000	3.558920e+05
max	10.000000	13504.000000	10.000000	2.436536e+07

Hình 1.4. Thống kê mô tả cho dữ liệu

Từ kết quả thống kê mô tả, có thể thấy biến Rating có giá trị trung bình khoảng 8.21 với độ lệch chuẩn 0.81, cho thấy mức đánh giá giữa các khách sạn không biến động quá lớn. Giá trị nhỏ nhất và lớn nhất lần lượt là 1 và 10, tuy nhiên trung vị ở mức 8.3 cho thấy phần lớn các khách sạn tập trung ở nhóm có điểm đánh giá cao. Điều này phản ánh xu hướng chung rằng khách hàng thường có trải nghiệm khá tích cực đối với các khách sạn trong bộ dữ liệu.

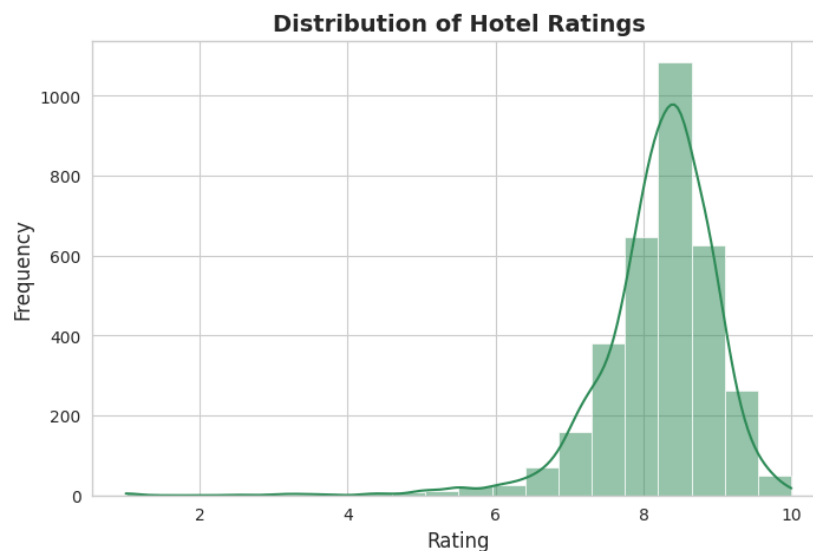
Đối với Number of Reviews, mặc dù giá trị trung bình đạt khoảng 1,046 lượt, nhưng độ lệch chuẩn lớn (1362,79), dữ liệu phân tán mạnh. Ngoài ra, có thể thấy một số khách sạn thu hút lượng đánh giá rất lớn, trong khi nhiều khách sạn khác chỉ có số lượng đánh giá hạn chế. Sự chênh lệch này liên quan đến mức độ phổ biến, vị trí hoặc quy mô hoạt động của từng khách sạn, thay vì chỉ phản ánh chất lượng dịch vụ.

Biến Room Score có giá trị trung bình khoảng 8.78 và độ lệch chuẩn tương đối thấp (0.47), cho thấy đánh giá về chất lượng phòng khá đồng đều giữa các khách sạn. Các giá trị chủ yếu nằm trong khoảng từ 8.4 đến 9.1, cho thấy khách hàng nhìn chung

có xu hướng hài lòng với yếu tố này, và đây có thể là một trong những điểm mạnh ổn định của dịch vụ.

Ngược lại, Room Price lại có mức biến động rất lớn. Dù giá trung bình khoảng 343,707, nhưng độ lệch chuẩn cao (748,824) cùng với khoảng giá trải dài từ 2,588 đến hơn 24 triệu cho thấy sự phân hóa rõ rệt giữa các phân khúc khách sạn. Điều này cho thấy tập dữ liệu không chỉ bao gồm các khách sạn cùng mức chất lượng, mà trải rộng từ bình dân đến cao cấp.

Các chỉ số cho thấy phần lớn khách sạn có đánh giá tích cực, đặc biệt ở khía cạnh chất lượng phòng. Tuy nhiên, sự khác biệt lớn về giá và số lượng đánh giá cho thấy thị trường có sự đa dạng đáng kể về quy mô và phân khúc.

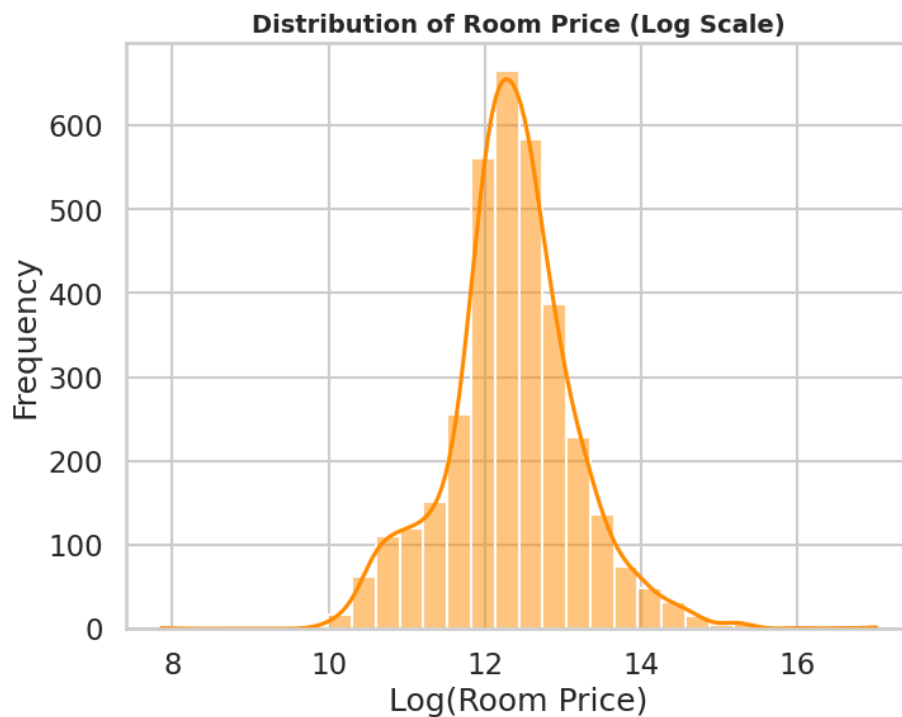


Hình 1.5. Phân bố điểm đánh giá khách sạn (Rating)

Phân bố của biến *Rating* cho thấy phần lớn các khách sạn đạt điểm trong khoảng 7.5 đến 9.0, với vùng tập trung rõ rệt quanh mức 8.2 - 8.5. Nhìn vào mật độ này, có thể nhận ra được là các đánh giá tích cực chiếm ưu thế trong tập dữ liệu, thay vì phân tán đều trên toàn bộ thang điểm.

Những trường hợp có điểm thấp (dưới 6) xuất hiện rất ít và nằm rải rác, không tạo thành cụm đáng kể. Điều này thể hiện rằng dữ liệu đang phản ánh nhóm khách sạn đã được chọn lọc ở một mức chất lượng nhất định, hoặc các khách sạn có trải nghiệm kém, ít nhận được sự chú ý từ người dùng.

Xét về hình dạng tổng thể, phân bố nghiêng nhẹ về phía trái khi phần đuôi kéo dài ở vùng điểm thấp. Đa số quan sát nằm ở phía điểm cao, còn các giá trị thấp chỉ chiếm tỷ trọng nhỏ. Kiểu phân bố này thường xuất hiện trong các nền tảng đánh giá, người dùng có xu hướng để lại phản hồi khi trải nghiệm đủ tốt, trong khi các trường hợp tiêu cực có thể bị thiếu hụt hoặc phân tán.



Hình 1.6. Phân bố giá phòng (Room Price)

Do giá phòng có độ chênh lệch rất lớn giữa các khách sạn từ mức giá thấp đến cao, dẫn đến các phân bố ban đầu bị lệch phải mạnh và chịu ảnh hưởng lớn từ các giá trị cực đại. Vì thế biến Room Price được áp dụng phép biến đổi log để làm cho dữ liệu ổn định và dễ quan sát hơn.

Cụ thể, phần lớn các quan sát nằm trong khoảng từ 11 đến 13, với mật độ cao nhất xung quanh vùng 12 - 12.5. Điều này cho thấy phần lớn khách sạn có mức giá tương đối gần nhau khi xét trên thang đo log, thay vì phân tán mạnh như ở dữ liệu gốc.

Phân bố vẫn giữ một phần đuôi kéo dài về phía bên phải, phản ánh sự tồn tại của một nhóm nhỏ khách sạn có giá cao hơn đáng kể so với mặt bằng chung. Tuy nhiên, sau khi biến đổi log, ảnh hưởng của các giá trị này đã được giảm bớt, giúp dữ liệu trở nên cân đối hơn.

CHƯƠNG 2: GIỚI THIỆU BÀI TOÁN

2.1. Phát biểu bài toán

Dữ liệu được thu thập từ nền tảng Booking.com phản ánh trải nghiệm thực tế của khách du lịch khi sử dụng dịch vụ lưu trú. Thông qua các đánh giá này, có thể nhận diện được mức độ hài lòng cũng như hiểu được quan điểm đánh giá của khách hàng về chất lượng dịch vụ mà khách sạn cung cấp.

Tuy nhiên, khi số lượng đánh giá ngày càng gia tăng trên các nền tảng trực tuyến, việc đọc và phân tích thủ công không còn là phương án khả thi. Quá trình này vừa tốn thời gian, vừa khó đảm bảo tính nhất quán trong việc tổng hợp thông tin. Vì vậy, thật là quan trọng để áp dụng các phương pháp phân tích dữ liệu kết hợp với học máy trở nên cần thiết để khai thác hiệu quả nguồn dữ liệu lớn này. Các mô hình học máy có khả năng nhận diện các mẫu trong dữ liệu và liên kết chúng với mức độ hài lòng của khách hàng, từ đó hỗ trợ tự động hóa quá trình phân tích.

Trên cơ sở đó, nghiên cứu hướng đến việc xây dựng một mô hình có khả năng phân loại mức độ hài lòng của khách du lịch dựa trên các thông tin liên quan đến khách sạn và nội dung đánh giá. Mô hình sau khi được huấn luyện không chỉ phục vụ cho việc phân loại dữ liệu hiện tại mà còn có thể áp dụng để dự đoán cho các đánh giá mới trong tương lai. Kết quả thu được góp phần làm rõ đặc điểm của dữ liệu đánh giá khách sạn, đồng thời cung cấp thêm cơ sở cho việc phân tích trải nghiệm khách hàng trong lĩnh vực du lịch và dịch vụ lưu trú.

2.2. Loại bài toán, biến đầu vào và biến mục tiêu

2.2.1. Phân loại nhị phân trong học máy

Trong lĩnh vực học máy, bài toán phân loại nhị phân (Binary Classification) là một dạng của học có giám sát, trong đó mục tiêu là phân loại dữ liệu vào một trong hai nhóm khác nhau. Hai nhóm này thường được biểu diễn dưới dạng nhãn 0 và 1, hoặc các trạng thái như đúng/sai, có/không, hài lòng/không hài lòng. Mục tiêu của phân loại nhị phân là dự báo xác suất xảy ra hoặc vắng mặt của sự kiện lần lượt tương ứng với nhãn 1 và 0. Tổng của hai xác suất này bằng 1:

$$P(y = 1|x, w) + P(y = 0|x, w) = 1$$

Trong công thức trên thì $P(y = 1|x)$ là xác suất có điều kiện, nó thể hiện xác suất của tích cực tại một quan sát x ứng với một mô hình cụ thể có trọng số w . Giá trị của xác suất này nằm trong khoảng $[0,1]$.

Mỗi đối tượng dữ liệu được biểu diễn thông qua một tập hợp các đặc trưng. Các đặc trưng này tạo thành không gian đặc trưng, nơi mà mô hình học máy tìm cách xác định ranh giới phân loại giữa hai nhóm dữ liệu. Ranh giới này có thể là tuyến tính hoặc phi tuyến tùy thuộc vào thuật toán được sử dụng. Khi có một dữ liệu mới được đưa vào hệ thống, mô hình sẽ dựa vào ranh giới phân loại đã học được để xác định dữ liệu đó thuộc về lớp nào.

Có nhiều thuật toán khác nhau có thể được sử dụng để giải quyết bài toán phân loại nhị phân. Một số thuật toán phổ biến bao gồm Logistic Regression, Support Vector Machine, Decision Tree và Random Forest. Mỗi thuật toán có cách tiếp cận khác nhau trong việc xây dựng mô hình và tìm ranh giới phân loại, tuy nhiên tất cả đều hướng tới mục tiêu chung là dự đoán chính xác nhãn của dữ liệu dựa trên các đặc trưng đầu vào. Các mô hình sau khi được huấn luyện thường sẽ được đánh giá thông qua các chỉ số như accuracy, precision, recall và F1-score để xác định mức độ hiệu quả của mô hình.

2.2.2. Biến đầu vào và biến mục tiêu

Câu lệnh nhằm thực hiện chuẩn hóa và tạo biến mục tiêu như sau:

```
df["Review Score"] = df["Review Score"].str.strip()
df["Satisfaction"] = df["Review Score"].apply(
    lambda x: 1 if x in ["Very Good", "Excellent", "Wonderful"] else 0)
```

	Review Score	Satisfaction
0	Very Good	1
1	Good	0
2	Very Good	1
3	Good	0
4	Wonderful	1

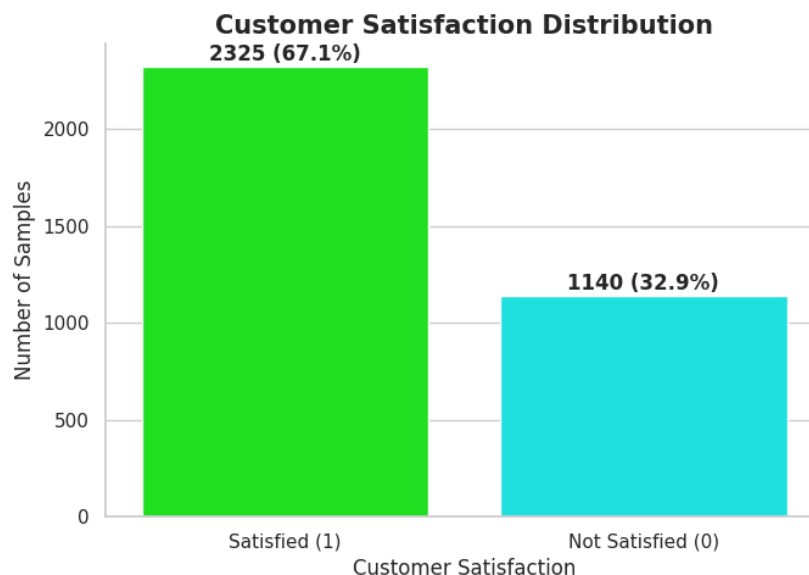
Hình 2.1. Kết quả thực hiện chuẩn hóa và tạo biến mục tiêu

Biến mục tiêu Satisfaction được xây dựng nhằm biểu diễn mức độ hài lòng của khách hàng đối với khách sạn. Biến này được suy ra từ cột Review Score trong bộ dữ liệu, vốn chứa các mức đánh giá dạng văn bản.

Trước khi tiến hành chuyển đổi, dữ liệu trong cột Review Score được làm sạch bằng cách loại bỏ các khoảng trắng dư thừa thông qua hàm `str.strip()`, giúp đảm bảo tính nhất quán về định dạng. Bước xử lý này đơn giản nhưng cần thiết để tránh sai lệch trong quá trình ánh xạ giá trị.

Biến Satisfaction được tạo bằng cách áp dụng hàm `apply()` kết hợp với biểu thức lambda để thực hiện phân loại nhị phân. Các mức đánh giá như “Very Good”, “Excellent” và “Wonderful” được xem là phản hồi tích cực và được gán giá trị 1. Ngược lại, những đánh giá còn lại, bao gồm “Good” và các mức thấp hơn, được quy về giá trị 0.

Sau bước chuyển đổi, dữ liệu được đưa về dạng nhị phân, trong đó mỗi quan sát chỉ nhận một trong hai giá trị 0 hoặc 1. Cách làm này giúp đơn giản hóa bài toán và phù hợp với các mô hình phân loại trong giai đoạn huấn luyện.



Hình 2.2. Biểu đồ phân bố biến mục tiêu trong tập dữ liệu

Kết quả thể hiện số lượng khách hàng hài lòng (Satisfied = 1) chiếm phần lớn với 2325 mẫu, tương đương 67.1% tổng số quan sát. Trong khi đó, nhóm không hài lòng (Not Satisfied = 0) có 1140 mẫu, chiếm 32.9%.

Đây là bộ dữ liệu có sự chênh lệch và cho thấy được đa số các khách hàng đều đánh giá tích cực về sản phẩm hoặc dịch vụ. Tuy nhiên, tỷ lệ khách hàng không hài lòng vẫn chiếm gần 1/3 tổng số mẫu.

Phân bố dữ liệu giữa hai lớp không hoàn toàn cân bằng nhưng không quá lớn. Như vậy, các mô hình phân loại vẫn học được đặc trưng của cả hai lớp, đồng thời đảm bảo khả năng dự đoán mức độ hài lòng của khách hàng một cách hiệu quả.

Trong quá trình xây dựng mô hình, các biến đầu vào được lựa chọn từ tập dữ liệu nhằm cung cấp thông tin để mô hình dự đoán mức độ hài lòng của khách hàng.

Các biến đầu vào bao gồm:

- + Rating: Điểm đánh giá tổng thể của khách hàng đối với khách sạn hoặc dịch vụ.
- + Number of Reviews: Số lượng người hoặc số lượng đặt phòng liên quan đến lượt đánh giá.
- + Room Price (in BDT or any other currency): Giá phòng được ghi nhận trong dữ liệu, thể hiện chi phí mà khách hàng phải trả cho dịch vụ.
- + Room Type: Loại phòng mà khách hàng đã đặt, ví dụ như phòng đơn, phòng đôi hoặc phòng cao cấp.
- + Bed Type: Loại giường trong phòng, chẳng hạn như giường đơn hoặc giường đôi.
- + Location: Vị trí của khách sạn, yếu tố có thể ảnh hưởng đến trải nghiệm và mức độ hài lòng của khách hàng.

2.3. Mô hình sử dụng và quy trình thực hiện

2.3.1. Mô hình sử dụng

Bài nghiên cứu này sử dụng hai thuật toán học có giám sát để xây dựng mô hình phân loại mức độ hài lòng của khách hàng, bao gồm Logistic Regression và Support Vector Machine (SVM). Hai mô hình này được lựa chọn vì đặc thù của bài toán là phân loại nhị phân, trong đó biến mục tiêu được biểu diễn dưới dạng hai trạng thái rời rạc: hài lòng (1) và không hài lòng (0). Đồng thời, đây cũng là hai phương pháp đại diện cho

hai hướng tiếp cận khác nhau trong học máy, từ đó có thể đánh giá một cách toàn diện hơn về hiệu năng mô hình.

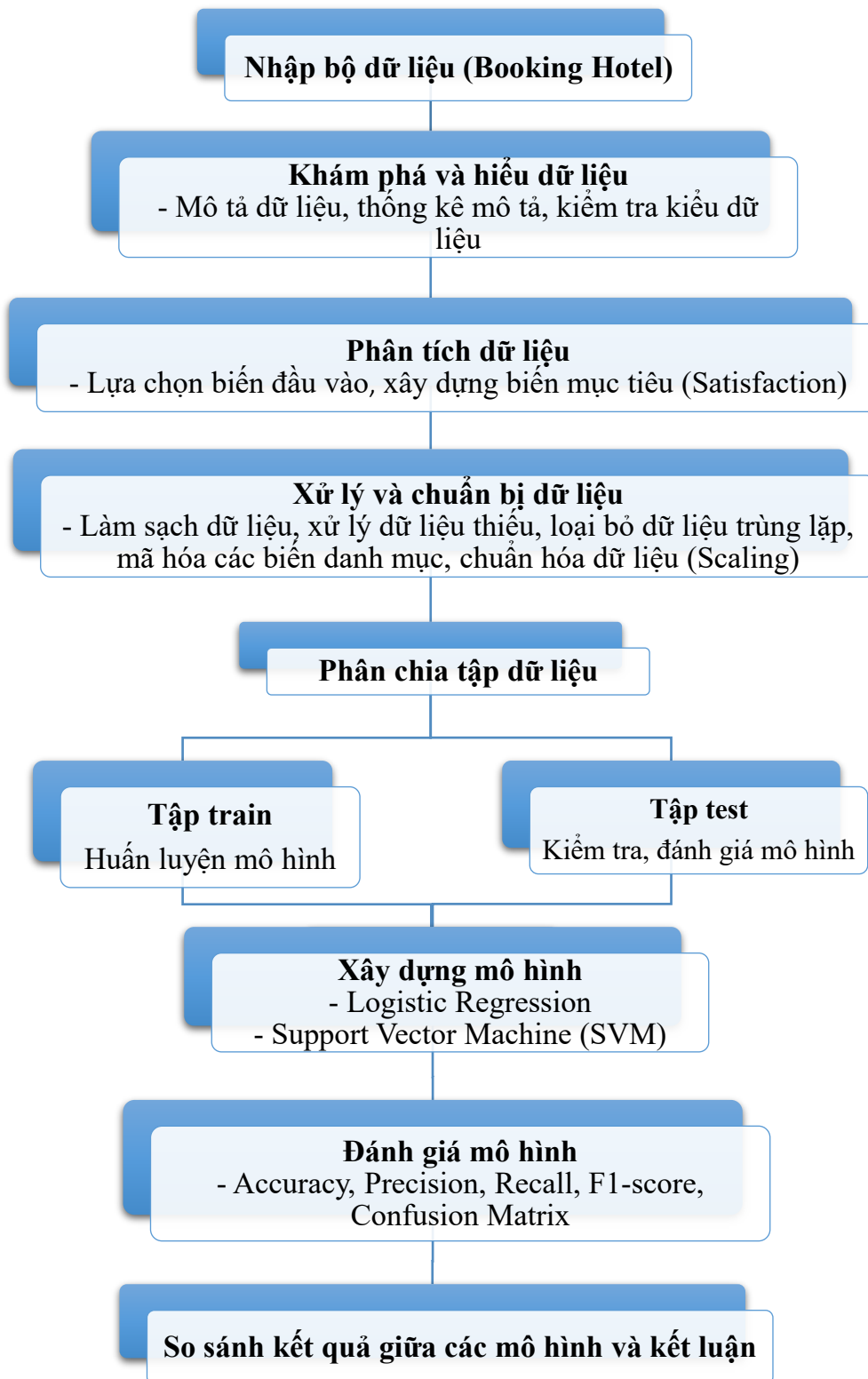
Đối với Logistic Regression, mô hình được sử dụng để thiết lập đường cơ sở cho quá trình so sánh. Về bản chất, Logistic Regression không trực tiếp dự đoán nhãn mà ước lượng xác suất một quan sát thuộc về lớp dương thông qua hàm sigmoid. Nhờ cơ chế này, đầu ra của mô hình được giới hạn trong khoảng $[0,1]$, tạo điều kiện thuận lợi cho việc diễn giải kết quả dưới dạng xác suất. Việc phân loại sau đó được thực hiện thông qua một ngưỡng quyết định, thường được đặt ở mức 0.5, mặc dù trong thực tế ngưỡng này có thể được điều chỉnh tùy theo mục tiêu tối ưu. Một điểm đáng chú ý là các tham số của Logistic Regression có thể được diễn giải trực tiếp thông qua mối quan hệ giữa đặc trưng đầu vào và log - odds của xác suất.

Tuy nhiên, do bản chất tuyến tính, Logistic Regression có thể gặp hạn chế khi ranh giới phân loại giữa các lớp không thể biểu diễn tốt bằng một siêu phẳng trong không gian đặc trưng ban đầu. Trong khi đó, Support Vector Machine được sử dụng như một phương pháp thay thế nhằm cải thiện khả năng phân tách dữ liệu. SVM không tập trung vào việc mô hình hóa xác suất mà trực tiếp tìm kiếm một siêu phẳng tối ưu sao cho khoảng cách biên giữa hai lớp được tối đa hóa. Đây là một cách giúp tăng cường khả năng khái quát hóa của mô hình, đặc biệt khi dữ liệu có nhiễu hoặc các điểm nằm gần ranh giới phân loại.

Bên cạnh đó, thông qua việc sử dụng các hàm kernel, SVM có thể ánh xạ dữ liệu từ không gian ban đầu sang một không gian có số chiều cao hơn. Nhờ vậy, mô hình có khả năng xử lý các mối quan hệ phi tuyến một cách hiệu quả mà các phương pháp tuyến tính khó nắm bắt.

Việc đưa cả hai mô hình vào thử nghiệm nhằm so sánh kết quả dự đoán đồng thời chọn ra được mô hình mang lại kết quả tốt cho đề tài nghiên cứu. Logistic Regression hoạt động như một điểm tham chiếu đơn giản, dễ theo dõi và ít tốn chi phí tính toán. Ngược lại, SVM được kỳ vọng xử lý tốt hơn trong trường hợp ranh giới giữa các lớp không rõ ràng. Từ sự khác biệt này, có thể đánh giá được mức độ phù hợp của từng mô hình và làm cơ sở cho việc lựa chọn hướng tiếp cận ở các bước tiếp theo.

2.3.2. Quy trình thực hiện



CHƯƠNG 3: XỬ LÝ DỮ LIỆU

3.1. Làm sạch dữ liệu và xử lý dữ liệu bị thiếu

```
df.isnull().sum()
```

	0
Hotel Name	0
Location	0
Rating	106
Review Score	106
Number of	137
Room Score	756
Room Type	0
Bed Type	16
Room Price (in BDT or any other currency)	0
Satisfaction	0

```
dtype: int64
```

Hình 3.1. Kiểm tra dữ liệu bị thiếu

Để xác định sự tồn tại của các giá trị thiếu (missing values), nghiên cứu sử dụng hàm `isnull().sum()` trong thư viện Pandas nhằm thống kê số lượng giá trị bị thiếu trong từng biến của bộ dữ liệu.

Kết quả kiểm tra cho thấy một số biến trong bộ dữ liệu có chứa dữ liệu thiếu.

- + Biến Rating có 106 giá trị thiếu
- + Biến Review Score có 106 giá trị thiếu
- + Biến Number of có 137 giá trị thiếu,
- + Biến Room Score có 756 giá trị thiếu
- + Biến Bed Type có 16 giá trị thiếu.

Các biến khác như Hotel Name, Location, Room Type, Room Price và Satisfaction không chứa giá trị thiếu.

Trong số các biến có dữ liệu thiếu, Room Score là biến có số lượng giá trị thiếu lớn nhất với 756 trường hợp. Những giá trị thiếu này có thể ảnh hưởng đến quá trình huấn luyện mô hình nếu không được xử lý phù hợp.

```
for col in df.select_dtypes(include="number").columns:
    df[col] = df[col].fillna(df[col].median())
for col in df.select_dtypes(include="object").columns:
    df[col] = df[col].fillna(df[col].mode()[0])
```

Bước tiếp theo trong quá trình làm sạch dữ liệu là xử lý các giá trị thiếu để đảm bảo tính đầy đủ và nhất quán của dữ liệu đầu vào. Thay vì loại bỏ các quan sát chứa giá trị thiếu, nghiên cứu áp dụng phương pháp điền giá trị dựa trên đặc trưng của từng kiểu biến.

Đối với các biến định lượng, giá trị thiếu được thay thế bằng trung vị (median) của từng cột tương ứng. Giá trị trung vị được chọn vì các biến số như giá phòng hoặc số lượng đánh giá thường có phân bố lệch và tồn tại các giá trị ngoại lai. So với trung bình (mean), trung vị ít bị ảnh hưởng bởi các giá trị cực trị, do đó giúp duy trì tốt hơn cấu trúc phân bố dữ liệu ban đầu.

Đối với các biến định tính, giá trị thiếu được xử lý bằng cách thay thế bằng giá trị xuất hiện nhiều nhất (mode) trong từng cột. Phương pháp này đảm bảo các giá trị được điền vẫn phản ánh xu hướng phổ biến của dữ liệu và hạn chế việc đưa vào các giá trị giả định không có ý nghĩa.

Về mặt thuật toán, quy trình xử lý được thực hiện bằng cách duyệt qua từng nhóm biến theo kiểu dữ liệu. Với mỗi biến định lượng x_j , giá trị thiếu được thay thế theo quy tắc:

$$x_j^{(i)} = \text{median}(x_j), \text{ nếu } x_j^{(i)} \text{ bị thiếu}$$

Trong khi đó, với mỗi biến định tính x_k , giá trị được cập nhật như sau:

$$x_k^{(i)} = \text{mode}(x_k), \text{ nếu } x_k^{(i)} \text{ bị thiếu}$$

Đây là cách xử lý dữ liệu thiếu một cách có hệ thống hơn và phù hợp với đặc trưng của từng loại biến, đồng thời giữ lại tối đa số lượng quan sát phục vụ cho các bước huấn luyện mô hình tiếp theo.

```
df.isnull().sum()
```

	0
Hotel Name	0
Location	0
Rating	0
Review Score	0
Number of	0
Room Score	0
Room Type	0
Bed Type	0
Room Price (in BDT or any other currency)	0
Satisfaction	0

dtype: int64

Hình 3.2. Kiểm tra dữ liệu sau khi xử lý

Như vậy, sau khi xử lý các giá trị bị thiếu và kiểm tra lại bằng lệnh `isnull().sum()`, kết quả trả về bằng 0 đối với tất cả các biến của tập dữ liệu.

```
# Các cột không dùng cho mô hình
unused_cols = ["Hotel Name", "Review Score"]
df.drop(columns=[col for col in unused_cols if col in df.columns],
         inplace=True)
```

Một số biến không cần thiết cho việc xây dựng mô hình nên đã được loại bỏ khỏi tập dữ liệu. Cụ thể, hai biến `Hotel Name` và `Review Score` được loại bỏ trước khi tiến hành huấn luyện mô hình. Biến `Hotel Name` chỉ mang tính chất định danh của khách sạn và không cung cấp thông tin có ý nghĩa cho việc dự đoán mức độ hài lòng của khách hàng, do đó không có giá trị trong quá trình học của mô hình. Bên cạnh đó, biến `Review Score` đã được sử dụng trước đó để tạo ra biến mục tiêu `Satisfaction`. Nếu tiếp tục giữ lại biến này trong tập dữ liệu đầu vào, mô hình có thể sử dụng trực tiếp thông tin đã dùng để tạo nhãn dự đoán, dẫn đến hiện tượng data leakage và làm sai lệch kết quả huấn luyện.

Để loại bỏ các biến đó, một danh sách các biến không sử dụng (`unused_cols`) được khai báo, bao gồm “`Hotel Name`” và “`Review Score`”.

Sau đó, chương trình thực hiện kiểm tra từng phần tử trong danh sách này thông qua một biểu thức lặp. Với mỗi cột trong `unused_cols`, hệ thống sẽ xác định xem cột đó có tồn tại trong `DataFrame` hay không. Nếu tồn tại, cột tương ứng sẽ được đưa vào danh sách các cột cần loại bỏ.

Cuối cùng, hàm `drop()` được áp dụng để xóa các cột đã xác định khỏi `DataFrame`. Tham số `inplace=True` cho phép cập nhật trực tiếp trên dữ liệu ban đầu mà không cần tạo bản sao mới.

3.2. Mã hóa biến phân loại và chuẩn hóa biến số

Để thực hiện mã hóa các biến phân loại và chuẩn hóa biến số, ta thực hiện đoạn mã sau:

```
# Mã hóa biến phân loại & chuẩn hóa biến số
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer

numeric_features = [
    "Rating",
    "Number of",
    "Room Price (in BDT or any other currency)"
]

categorical_features = [
    "Room Type",
    "Bed Type",
    "Location"
]

preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numeric_features),
        ("cat", OneHotEncoder(handle_unknown="ignore"),
categorical_features)
    ]
)
```

Các biến được tách thành hai nhóm chính dựa trên kiểu dữ liệu: biến số và biến phân loại. Đây là cách nhằm giúp áp dụng đúng phương pháp biến đổi cho từng loại đặc trưng.

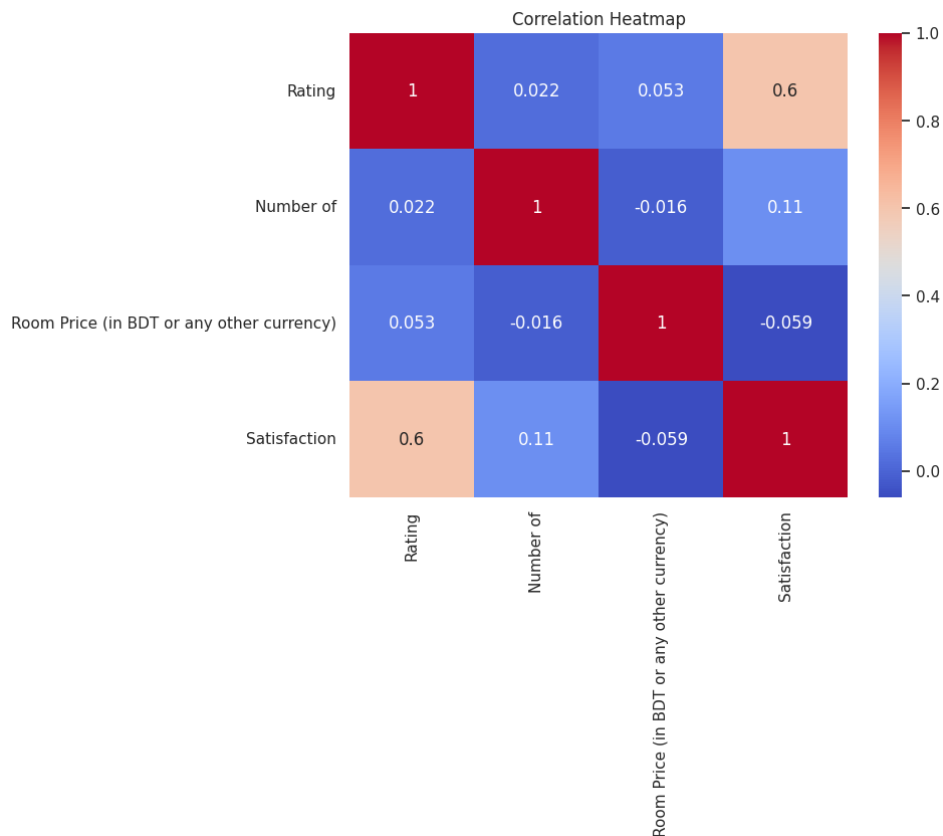
Nhóm biến số gồm các thuộc tính như `Rating`, `Number of Reviews` và `Room Price`. Các biến này được đưa qua `StandardScaler`, tức là mỗi cột được chuẩn hóa dựa trên giá

trị trung bình và độ lệch chuẩn của chính nó. Sau bước này, dữ liệu được đưa về cùng thang đo, hạn chế tình trạng một số biến có giá trị lớn chi phối quá trình học.

Với các biến dạng phân loại như Room Type, Bed Type và Location, dữ liệu được chuyển sang dạng số bằng One-Hot Encoding. Mỗi giá trị khác nhau trong một cột sẽ được biểu diễn thành một biến nhị phân riêng. Cách này giúp mô hình xử lý dữ liệu dạng chữ mà không giả định thứ tự giữa các nhóm.

Các bước xử lý trên được thực hiện gộp thông qua ColumnTransformer. Đây là công cụ áp dụng nhiều phép biến đổi khác nhau lên từng nhóm cột trong cùng một pipeline. Nhờ vậy, dữ liệu đầu vào luôn được xử lý theo cùng một cách ở cả giai đoạn huấn luyện và kiểm định, đồng thời thuận tiện khi kết nối trực tiếp với các mô hình phía sau.

3.3. Phân tích tương quan và chia tập dữ liệu



Hình 3.3. Ma trận tương quan giữa các biến trong tập dữ liệu

- Từ ma trận tương quan trên:

+ Rating là biến có mối liên hệ rõ ràng nhất với Satisfaction, với hệ số khoảng 0.6. Mức tương quan này cho thấy khi điểm đánh giá tổng thể tăng, mức độ hài lòng của khách hàng cũng có xu hướng tăng theo. Đây là kết quả rất hợp lý vì Rating là biến đã phản ánh trải nghiệm tổng hợp của người dùng.

+ Number of gần như không thể hiện mối quan hệ đáng kể với các biến còn lại. Hệ số tương quan với Rating chỉ khoảng 0.022 và với Satisfaction khoảng 0.11, các con số này cho thấy số lượng đánh giá không phản ánh trực tiếp chất lượng hay mức độ hài lòng. Nói cách khác, một khách sạn có nhiều lượt đánh giá chưa chắc đã được đánh giá cao hơn.

+ Room Price có hệ số tương quan với Satisfaction ở mức - 0.059, cho thấy mối liên hệ rất yếu và gần như không đáng kể. Dữ liệu cho thấy rằng dù giá cao thì cũng không đồng nghĩa với việc khách hàng hài lòng hơn. Các trải nghiệm dịch vụ, tiện nghi hoặc vị trí có thể đóng vai trò quan trọng hơn so với yếu tố giá.

Nhìn chung, ngoài mối liên hệ tương đối rõ giữa Rating và Satisfaction, các biến còn lại có mức tương quan thấp. Điều này giúp giảm khả năng xuất hiện đa cộng tuyến khi đưa vào mô hình, mỗi biến có thể đóng góp thông tin theo những cách khác nhau thay vì bị trùng lặp.

• Chia tập dữ liệu Train/Test:

```
# Chia tập dữ liệu train-test
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.3,
    random_state=42,
    stratify=y
)
```

Sau khi hoàn tất bước tiền xử lý, dữ liệu được tách thành hai phần để huấn luyện và đánh giá mô hình. Việc phân chia này được thực hiện thông qua hàm `train_test_split` trong thư viện Scikit-learn.

Tập dữ liệu gốc ban đầu được chia theo tỷ lệ 70 - 30, trong đó 70% số quan sát được sử dụng để huấn luyện mô hình, phần còn lại dùng để kiểm tra khả năng dự đoán trên dữ liệu chưa từng xuất hiện. Tham số `random_state` cố định ở giá trị 42 nhằm đảm bảo rằng mỗi lần chạy lại chương trình đều cho ra cùng một cách chia dữ liệu, từ đó giúp kết quả có thể so sánh và tái lập.

Một điểm cần lưu ý trong quá trình tách dữ liệu là phân bố của biến mục tiêu. Thay vì chia ngẫu nhiên hoàn toàn, ta sử dụng tham số `stratify = y` để giữ nguyên tỷ lệ các lớp giữa hai tập dữ liệu. Nhờ đó, mô hình được huấn luyện trên một tập dữ liệu có cấu trúc tương tự với tập kiểm tra, hạn chế tình trạng một lớp bị thiếu hụt hoặc chiếm ưu thế quá mức trong từng tập.

Hàm sẽ nhận đầu vào là ma trận đặc trưng X và vector nhãn y , sau đó trả về bốn thành phần: $X_{\text{train}}, X_{\text{test}}, y_{\text{train}}, y_{\text{test}}$. Đây là các tập dữ liệu được sử dụng xuyên suốt các bước huấn luyện và đánh giá mô hình ở giai đoạn tiếp theo.

CHƯƠNG 4: MÔ HÌNH HÓA DỮ LIỆU

4.1. Mô hình Logistic Regression

4.1.1. Cơ sở lý thuyết

Logistic Regression là một trong những mô hình phân loại phổ biến và được sử dụng rộng rãi trong thống kê cũng như học máy, đặc biệt đối với các bài toán phân loại nhị phân. Mô hình có dạng tổng quát $\mathbf{y} = \mathbf{f}(\mathbf{w}^T \mathbf{x})$. Mục tiêu của mô hình là ước lượng xác suất để một quan sát thuộc về một lớp nhất định thông qua hàm liên kết logit. Theo Jurafsky và Martin, Logistic Regression được xem là một mô hình nền tảng trong các bài toán phân loại và có mối liên hệ nhất định với nhiều mô hình học sâu hiện đại.

Trong bài toán phân loại nhị phân, mô hình được sử dụng để ước lượng xác suất $P(Y = 1 | X)$, trong đó X là vector đặc trưng và Y là biến phụ thuộc dạng định tính. Do xác suất chỉ nằm trong khoảng từ 0 đến 1 nên việc áp dụng hồi quy tuyến tính thông thường sẽ không phù hợp. Vì vậy, Logistic Regression sử dụng hàm logit để thiết lập mối quan hệ tuyến tính giữa biến phụ thuộc và các biến độc lập. Hàm logit được định nghĩa là $\text{Logit}(p) = \log\left(\frac{p}{1-p}\right)$, cho phép chuyển đổi xác suất thành giá trị trên toàn bộ trục số thực. Hàm số được sử dụng phổ biến nhất là hàm sigmoid (hay logistic function) với công thức $\sigma(s) = \frac{1}{1+e^{-s}}$. Hàm này có các đặc tính quan trọng như liên tục, nhận giá trị trong khoảng (0,1), và có đạo hàm tại mọi nơi với công thức đơn giản $\sigma'(s) = \sigma(s)(1 - \sigma(s))$, điều này rất thuận lợi cho quá trình tối ưu hóa toán học.

Để tìm tham số $\mathbf{w}$ phù hợp nhất, mô hình áp dụng phương pháp Maximum Likelihood Estimation nhằm tìm bộ tham số sao cho xác suất các điểm dữ liệu rơi vào đúng class của chúng là cao nhất. Bằng cách giả định các điểm dữ liệu là độc lập và sử dụng logarit tự nhiên để chuyển tích thành tổng, chúng ta thu được hàm mất mát Negative Log-Likelihood, còn được gọi là Cross Entropy. Hàm này đo lường khoảng cách giữa phân phối xác suất do mô hình dự đoán và phân phối thực tế của dữ liệu. Việc tối ưu hóa thường được thực hiện bằng thuật toán Stochastic Gradient Descent. Nhờ cấu trúc đặc biệt của hàm sigmoid, công thức cập nhật trọng số được thiết lập một cách đơn giản hơn: $\mathbf{w} = \mathbf{w} + \mathbf{n}(\mathbf{y}_i - \mathbf{z}_i)$, trong đó $\mathbf{z}_i$ là xác suất dự đoán và $\mathbf{y}_i$ là nhãn thực tế.

Về mặt tính chất, mặc dù mang tên là hồi quy, Logistic Regression lại được ứng dụng rộng rãi nhất trong các bài toán phân lớp (Classification). Ranh giới phân chia giữa các class (Decision Boundary) của mô hình này luôn có dạng tuyến tính (một siêu mặt phẳng), được xác định tại ngưỡng xác suất 0.5, tương ứng với phương trình $\mathbf{w}^T \mathbf{x} = 0$. Logistic Regression có ưu điểm là không yêu cầu dữ liệu phải phân tách tuyến tính hoàn toàn như PLA, nhưng nó vẫn gặp hạn chế với các tập dữ liệu có cấu trúc phi tuyến phức tạp và đòi hỏi giả định các điểm dữ liệu phải độc lập với nhau. Khi biểu diễn dưới dạng mạng nơ-ron, mô hình này có thể được xem như một nơ-ron đơn lẻ với hàm kích hoạt sigmoid ở đầu ra.

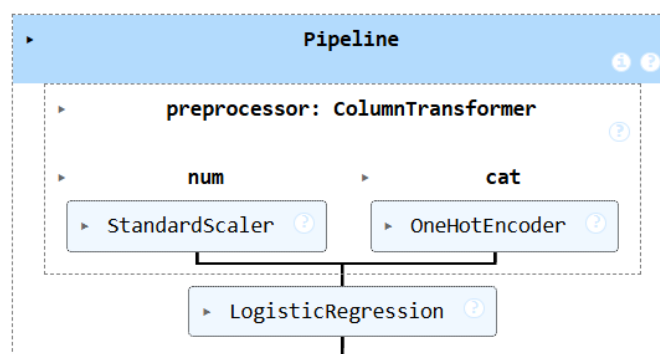
4.1.2. Áp dụng mô hình

Thực hiện huấn luyện dữ liệu bằng mô hình Logistic Regression:

```
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline

logistic_model = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        ("classifier", LogisticRegression(max_iter=1000))
    ]
)

logistic_model.fit(X_train, y_train)
```



Hình 4.1. Pipeline train dữ liệu bằng mô hình Logistic Regression

Đoạn mã trên mô tả cách xây dựng và huấn luyện mô hình Logistic Regression thông qua cơ chế Pipeline trong Scikit-learn, kết hợp toàn bộ các bước tiền xử lý và học mô hình trong một quy trình thống nhất.

Thay vì xử lý dữ liệu rồi mới đưa vào mô hình một cách tách rời, dùng pipeline để thực hiện tuần tự hai giai đoạn. Ở bước đầu tiên, dữ liệu đầu vào đi qua khối preprocessor, được xây dựng bằng ColumnTransformer. Tại đây, các biến số được chuẩn hóa bằng StandardScaler, các biến phân loại được mã hóa bằng One-Hot Encoding. Toàn bộ quá trình biến đổi diễn ra tự động mỗi khi dữ liệu được đưa vào pipeline.

Sau khi hoàn tất bước biến đổi đặc trưng, dữ liệu được chuyển sang mô hình Logistic Regression. Thuật toán này ước lượng xác suất một quan sát thuộc lớp dương thông qua hàm sigmoid:

$$P(y = 1 | x) = \frac{1}{1 + e^{-(w^T x + b)}}$$

Trong quá trình huấn luyện, các tham số w và b được tối ưu bằng cách cực tiểu hóa hàm mất mát log-loss. Thiết lập tham số `max_iter=1000` nhằm tăng số vòng lặp tối đa cho thuật toán tối ưu, đảm bảo quá trình hội tụ khi dữ liệu có số chiều lớn sau bước mã hóa.

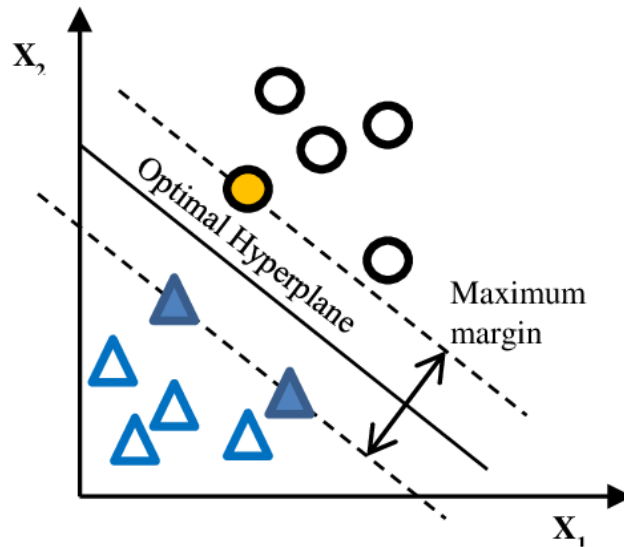
Khi áp dụng lệnh `fit(X_train, y_train)` với `X_train` là tập dữ liệu đặc trưng dùng để huấn luyện và `y_train` là biến mục tiêu tương ứng. Khi bắt đầu thực hiện, pipeline hoạt động bằng cách dữ liệu huấn luyện được đưa qua bước tiền xử lý để chuyển đổi sang dạng số phù hợp, sau đó kết quả được chuyển trực tiếp vào mô hình Logistic Regression để tiến hành học. Toàn bộ quy trình này được gộp thành một cụm duy nhất, do đó không cần gọi riêng lẻ từng bước xử lý.

4.2. Mô hình Support Vector Machine

4.2.1. Cơ sở lý thuyết

Support Vector Machine (SVM) là một trong những phương pháp phân loại mạnh mẽ trong học máy, được xây dựng dựa trên nguyên lý tối đa hóa khoảng biên (margin) giữa các lớp dữ liệu. Khác với mô hình Logistic Regression, SVM không trực tiếp ước lượng xác suất của từng lớp mà tìm một siêu phẳng (hyperplane) tối ưu để phân tách các lớp dữ liệu sao cho khoảng cách từ siêu phẳng đến các điểm dữ liệu gần nhất của mỗi lớp là lớn nhất. Phương pháp này được phát triển bởi Vapnik (1963) cho các bài toán phân loại nhị phân và sau đó được mở rộng để xử lý các bài toán phân loại đa lớp. Một ưu điểm nổi bật của SVM là khả năng làm việc hiệu quả với dữ liệu có số chiều

lớn, do đó thường được áp dụng trong nhiều lĩnh vực như khai phá dữ liệu, xử lý văn bản và nhận dạng mẫu.



Hình 4.2. Minh họa siêu phẳng tối ưu và khoảng biên trong SVM

Xét một bài toán phân loại nhị phân với tập dữ liệu huấn luyện $(\mathbf{x}_i, \mathbf{y}_i)$, trong đó $\mathbf{x}_i$ là vector đặc trưng và $\mathbf{y}_i \in \{-1, +1\}$ là nhãn của từng quan sát. Mục tiêu của SVM là tìm một siêu phẳng có dạng $\mathbf{w}^T \mathbf{x} + b = 0$ để phân chia không gian đặc trưng thành hai vùng tương ứng với hai lớp khác nhau. Khoảng cách từ một điểm dữ liệu đến siêu phẳng được xác định theo phương vuông góc với siêu phẳng đó. Lề (margin) của mô hình được định nghĩa là khoảng cách nhỏ nhất từ các điểm dữ liệu thuộc mỗi lớp đến siêu phẳng phân tách. SVM tìm các tham số $\mathbf{w}$ và b sao cho khoảng biên này đạt giá trị lớn nhất. Bài toán tối ưu được biểu diễn dưới dạng tối thiểu hóa chuẩn của vector trọng số $\|\mathbf{w}\|^2$ với các ràng buộc phân loại:

$$\mathbf{y}_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1$$

Việc xây dựng bài toán dưới dạng tối ưu lồi đảm bảo nghiệm thu được là nghiệm tối ưu toàn cục, từ đó giúp SVM tránh được hiện tượng rơi vào cực tiểu cục bộ thường gặp trong nhiều mô hình học máy phi tuyến.

Trong thực tế, dữ liệu thường không thể phân tách tuyến tính trong không gian đặc trưng ban đầu. Để giải quyết vấn đề này, SVM sử dụng hàm kernel nhằm ánh xạ dữ liệu từ không gian ban đầu sang một không gian đặc trưng có số chiều cao hơn, nơi các lớp

dữ liệu có thể phân tách tuyến tính. Trong nghiên cứu này, mô hình SVM được triển khai với Radial Basis Function kernel, đây là một trong những kernel phổ biến nhất trong học máy. Kernel RBF được định nghĩa như sau:

$$K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|_2^2), \gamma > 0$$

Trong đó γ là tham số điều chỉnh mức độ ảnh hưởng của từng điểm dữ liệu. Kernel RBF cho phép mô hình học được các ranh giới phân loại phi tuyến, giúp tăng khả năng phân loại trong các trường hợp dữ liệu có cấu trúc phức tạp. Nhờ đặc điểm này, SVM với kernel RBF thường đạt hiệu quả cao trong nhiều bài toán phân loại thực tế, đặc biệt khi mối quan hệ giữa các biến đầu vào và biến mục tiêu không mang tính tuyến tính.

4.2.2. Áp dụng mô hình

Thực hiện huấn luyện dữ liệu bằng mô hình SVM:

```
from sklearn.model_selection import GridSearchCV
svm_model = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        ("classifier", SVC(probability=True))
    ]
)
param_grid = {
    "classifier__C": [0.1, 1, 10],
    "classifier__gamma": ["scale", 0.1, 0.01],
    "classifier__kernel": ["rbf"]
}
grid = GridSearchCV(
    svm_model,
    param_grid,
    cv=5,
    scoring="accuracy"
)

grid.fit(X_train, y_train)
best_svm = grid.best_estimator_
```

SVM được triển khai thông qua thư viện **Scikit-learn** với lớp SVC. Giống như mô hình Logistic Regression, SVM cũng xây dựng dưới dạng Pipeline để đảm bảo dữ liệu được xử lý nhất quán trong quá trình huấn luyện và dự đoán.

Bên cạnh đó, để tối ưu hóa hiệu suất của mô hình, nghiên cứu sử dụng phương pháp GridSearchCV nhằm thực hiện quá trình thử nghiệm nhiều tổ hợp tham số khác nhau và đánh giá hiệu suất của từng tổ hợp thông qua kỹ thuật cross-validation. Phương pháp 5-fold cross-validation được áp dụng nghĩa là dữ liệu huấn luyện được chia thành 5 phần và mô hình sẽ được huấn luyện và đánh giá lặp lại 5 lần với các tập dữ liệu khác nhau.

Các tham số được tối ưu bao gồm:

- *C*: tham số điều chỉnh mức độ phạt khi mô hình phân loại sai. Giá trị *C* càng lớn thì mô hình càng cố gắng phân loại đúng các điểm dữ liệu huấn luyện.

- *gamma*: tham số của kernel RBF, kiểm soát phạm vi ảnh hưởng của từng điểm dữ liệu trong không gian đặc trưng.

- *kernel*: loại kernel được sử dụng trong SVM. Trong nghiên cứu này sử dụng Radial Basis Function (RBF) vì kernel này có khả năng mô hình hóa các ranh giới phân loại phi tuyến hiệu quả.

Quá trình tìm kiếm tham số được thực hiện trên các giá trị khác nhau của *C* và *gamma* nhằm xác định cấu hình mang lại độ chính xác cao nhất. Sau khi quá trình Grid Search hoàn tất, mô hình tốt nhất được lựa chọn thông qua thuộc tính `best_estimator_`, đây là mô hình có hiệu suất cao nhất trong quá trình đánh giá.

Sau khi thực hiện quá trình tìm kiếm tham số tối ưu bằng GridSearchCV, mô hình SVM thu được bộ tham số tốt nhất như sau:

Best parameters: `{'classifier__C': 10, 'classifier__gamma': 'scale', 'classifier__kernel': 'rbf'}`

Kết quả từ GridSearchCV cho thấy bộ tham số tối ưu của mô hình SVM là *C* = 10, *gamma* = "scale" và *kernel* = "rbf". Mô hình đạt hiệu suất tốt nhất khi sử dụng kernel RBF để học các ranh giới phân loại phi tuyến trong dữ liệu. Giá trị *C* = 10 giúp mô hình giảm sai số phân loại trên tập huấn luyện, trong khi *gamma* = "scale" tự động điều chỉnh phạm vi ảnh hưởng của các điểm dữ liệu dựa trên số lượng đặc trưng. Nhờ đó, mô hình có khả năng phân loại hiệu quả và cải thiện độ chính xác dự đoán.

CHƯƠNG 5: ĐÁNH GIÁ MÔ HÌNH

5.1. Các tiêu chí đánh giá

5.1.1. Ma trận nhầm lẫn

Ma trận nhầm lẫn là một trong những công cụ quan trọng để đánh giá mô hình phân loại, đặc biệt trong các bài toán nhị phân. Khác với các chỉ số tổng hợp chỉ phản ánh tỷ lệ dự đoán đúng trên toàn bộ dữ liệu, ma trận nhầm lẫn cung cấp cái nhìn chi tiết hơn về cách mô hình đưa ra dự đoán trên từng lớp cụ thể. Nhờ đó, người nghiên cứu có thể biết mô hình đúng bao nhiêu phần trăm đồng thời hiểu được mô hình sai ở đâu và sai theo xu hướng nào. Trong bài toán phân loại nhị phân, dữ liệu thường được chia thành hai lớp: Positive (nhãn 1) - lớp cần quan tâm hoặc phát hiện, và Negative (nhãn 0) - lớp còn lại. Từ đó, ma trận nhầm lẫn bao gồm bốn thành phần chính:

- True Positive (TP): số trường hợp thực tế thuộc lớp Positive và được mô hình dự đoán đúng là Positive.

- True Negative (TN): số trường hợp thực tế thuộc lớp Negative và mô hình cũng dự đoán đúng là Negative.

- False Positive (FP): số trường hợp thực tế thuộc lớp Negative nhưng bị mô hình dự đoán nhầm thành Positive. Lỗi này còn được gọi là báo động nhầm hoặc sai lầm loại I.

- False Negative (FN): số trường hợp thực tế thuộc lớp Positive nhưng lại bị dự đoán nhầm thành Negative. Đây là lỗi bỏ sót, còn gọi là sai lầm loại II.

Cách phân tách rõ bốn thành phần này giúp nhận diện được bản chất lỗi của mô hình. Nếu số lượng False Negative lớn, mô hình đang có xu hướng bỏ sót các trường hợp cần phát hiện, và ngược lại, nếu False Positive chiếm tỷ lệ cao, mô hình lại dễ đưa ra cảnh báo sai. Tùy vào mục tiêu của bài toán, mức độ nghiêm trọng của từng loại lỗi có thể khác nhau, do đó việc quan sát ma trận nhầm lẫn mang ý nghĩa thực tiễn rõ rệt hơn so với việc chỉ dựa vào một chỉ số duy nhất.

Ngoài cách biểu diễn bằng số lượng tuyệt đối, ma trận nhầm lẫn còn có thể được chuẩn hóa theo từng hàng. Mỗi giá trị trong một hàng được chia cho tổng số quan sát thực tế của lớp tương ứng, chuyển các giá trị về dạng tỷ lệ. Thay vì bị chi phối bởi lớp

có số lượng lớn, người phân tích có thể đánh giá được tỷ lệ dự đoán đúng và sai trên từng lớp một cách công bằng hơn.

5.1.2. Các chỉ số đánh giá

Từ các giá trị trong ma trận nhầm lẫn, ta có thể tính toán nhiều chỉ số khác nhau nhằm đánh giá mô hình theo nhiều khía cạnh.

- *Accuracy (Độ chính xác)*: Accuracy là chỉ số phổ biến nhất, thể hiện tỷ lệ giữa số dự đoán đúng và tổng số dự đoán của mô hình.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Tuy nhiên, Accuracy chỉ thực sự có ý nghĩa khi số lượng mẫu của các lớp tương đối cân bằng. Trong trường hợp dữ liệu bị lệch lớp, chỉ số này có thể gây hiểu lầm. Ví dụ, nếu 90% dữ liệu thuộc lớp GOOD và chỉ 10% thuộc lớp BAD, một mô hình dự đoán tất cả là GOOD vẫn đạt Accuracy 90%, nhưng thực tế lại không phát hiện được bất kỳ trường hợp BAD nào.

- *Precision*: Precision cho biết trong số các trường hợp mà mô hình dự đoán là Positive, có bao nhiêu trường hợp thực sự đúng.

$$Precision = \frac{TP}{TP + FP}$$

Chỉ số này đặc biệt quan trọng trong các tình huống mà việc dự đoán nhầm Positive gây hậu quả lớn. Chẳng hạn, trong hệ thống lọc email rác, việc nhầm một email quan trọng thành thư rác là điều không mong muốn.

- *Recall*: Recall đo lường khả năng của mô hình trong việc phát hiện các trường hợp Positive thực sự.

$$Recall = \frac{TP}{TP + FN}$$

Recall cho biết trong số tất cả các trường hợp thực sự là Positive, mô hình đã nhận diện được bao nhiêu phần trăm. Chỉ số này rất quan trọng trong những lĩnh vực mà việc bỏ sót một trường hợp là điều nghiêm trọng, chẳng hạn như chẩn đoán bệnh hoặc phát hiện rủi ro.

- *F1-score*: F1-score là chỉ số kết hợp giữa Precision và Recall thông qua trung bình điều hòa của hai giá trị này.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Việc sử dụng trung bình điều hòa giúp nhấn mạnh sự cân bằng giữa Precision và Recall. Nếu một trong hai chỉ số quá thấp thì F1-score cũng sẽ giảm đáng kể. Ví dụ, nếu Precision gần bằng 0 nhưng Recall rất cao, trung bình cộng vẫn có thể cho giá trị trung bình, nhưng F1-score sẽ phản ánh rõ rằng mô hình chưa hoạt động hiệu quả.

5.2. Kết quả và so sánh các mô hình

5.2.1. Kết quả từ Logistic Regression

• *Các chỉ số đánh giá:*

Accuracy (Logistic): 0.925					
	precision	recall	f1-score	support	
0	0.93	0.83	0.88	342	
1	0.92	0.97	0.95	698	
accuracy			0.93	1040	
macro avg	0.93	0.90	0.91	1040	
weighted avg	0.93	0.93	0.92	1040	

Kết quả đánh giá mô hình Logistic Regression đạt Accuracy = 0.925, tương đương khoảng 92.5% số quan sát trong tập kiểm tra được phân loại đúng. Điều này phản ánh khả năng dự đoán khá tốt của mô hình đối với bộ dữ liệu nghiên cứu.

Xét chi tiết theo từng lớp, lớp 0 (không hài lòng) có Precision = 0.93, Recall = 0.83 và F1-score = 0.88 với 342 quan sát. Giá trị Precision cao thể hiện một phần lớn các trường hợp được dự đoán là không hài lòng đều chính xác. Tuy nhiên, Recall thấp hơn phản ánh việc mô hình vẫn còn bỏ sót một số trường hợp thực sự không hài lòng.

Đối với lớp 1 (hài lòng), mô hình đạt Precision = 0.92, Recall = 0.97 và F1-score = 0.95 với 698 quan sát. Giá trị Recall rất cao thể hiện khả năng nhận diện phần lớn khách hàng hài lòng, đồng thời số lượng trường hợp bị bỏ sót ở lớp này khá thấp.

Ngoài ra, macro average F1-score = 0.91 và weighted average F1-score = 0.92 phản ánh hiệu suất tổng thể của mô hình trên toàn bộ tập dữ liệu. Do số lượng mẫu của lớp hài lòng (698) lớn hơn lớp không hài lòng (342), chỉ số weighted average chịu ảnh hưởng nhiều hơn từ lớp 1. Nhìn chung, Logistic Regression vẫn hoạt động hiệu quả trong việc phân loại mức độ hài lòng của khách hàng, mặc dù khả năng phát hiện đầy đủ các trường hợp không hài lòng vẫn có thể được cải thiện.

• *Ma trận nhầm lẫn:*

```
array([[284,  58],
       [ 20, 678]])
```

- Từ ma trận trên có thể phân tích:

+ True Negative (TN) = 284: Có 284 trường hợp thực tế thuộc lớp 0 và được mô hình dự đoán chính xác.

+ False Positive (FP) = 58: Có 58 trường hợp thực tế thuộc lớp 0 nhưng bị dự đoán nhầm thành lớp 1.

+ False Negative (FN) = 20: Có 20 trường hợp thực tế thuộc lớp 1 nhưng bị dự đoán nhầm thành lớp 0.

+ True Positive (TP) = 678: Có 678 trường hợp thực tế thuộc lớp 1 và được mô hình dự đoán đúng.

Tổng số quan sát trong tập kiểm tra là 1040 mẫu, trong đó mô hình dự đoán đúng 962 trường hợp, tương ứng với Accuracy $\approx 92.5\%$. Số lượng False Negative tương đối thấp (20 trường hợp) cho thấy mô hình hiếm khi bỏ sót các trường hợp thuộc lớp 1. Tuy nhiên, vẫn tồn tại 58 trường hợp False Positive, một số mẫu thuộc lớp 0 bị dự đoán nhầm thành lớp 1. Mô hình đang có xu hướng dự đoán lớp 1 nhiều hơn, phù hợp với việc lớp 1 chiếm tỷ lệ lớn hơn trong dữ liệu.

Nhìn chung, ma trận nhầm lẫn phản ánh rằng mô hình Logistic Regression đạt hiệu quả phân loại tốt, đặc biệt trong việc nhận diện các quan sát thuộc lớp 1.

5.2.2. Kết quả từ SVM

- *Các chỉ số đánh giá:*

Accuracy (SVM) : 0.9567307692307693

	precision	recall	f1-score	support
0	0.97	0.90	0.93	342
1	0.95	0.99	0.97	698
accuracy			0.96	1040
macro avg	0.96	0.94	0.95	1040
weighted avg	0.96	0.96	0.96	1040

Kết quả từ tập kiểm tra cho thấy mô hình SVM đạt độ chính xác khoảng 95.67%, tương ứng với 0.9567 trong chỉ số Accuracy. Đối với lớp 0, mô hình đạt Precision = 0.97 và Recall = 0.90, dẫn đến F1-score = 0.93. Điều này có nghĩa là hầu hết các trường hợp được dự đoán là lớp 0 đều chính xác, và khả năng nhận diện các quan sát thuộc lớp này cũng khá tốt.

Trong khi đó, lớp 1 đạt Precision = 0.95, Recall = 0.99 và F1-score = 0.97. Giá trị Recall gần 1 cho thấy mô hình hầu như nhận diện được tất cả các trường hợp thuộc lớp 1, do đó số lượng mẫu bị bỏ sót rất nhỏ.

Xét trên toàn bộ dữ liệu, macro average F1-score đạt 0.95 và weighted average F1-score đạt 0.96. Các chỉ số phản ánh hiệu suất phân loại đều giữa các lớp, ngay cả khi số lượng quan sát của mỗi lớp có sự khác biệt nhất định. Tổng thể, mô hình SVM đạt kết quả phân loại cao và phù hợp để áp dụng cho bài toán dự đoán trong nghiên cứu này.

- *Ma trận nhầm lẫn:*

```
array([[307, 35],
       [10, 688]])
```

- Từ ma trận trên có thể nhận thấy:

+ True Negative (TN) = 307: Có 307 trường hợp thực tế thuộc lớp 0 và được mô hình dự đoán đúng.

+ False Positive (FP) = 35: Có 35 trường hợp thực tế thuộc lớp 0 nhưng bị dự đoán nhầm thành lớp 1.

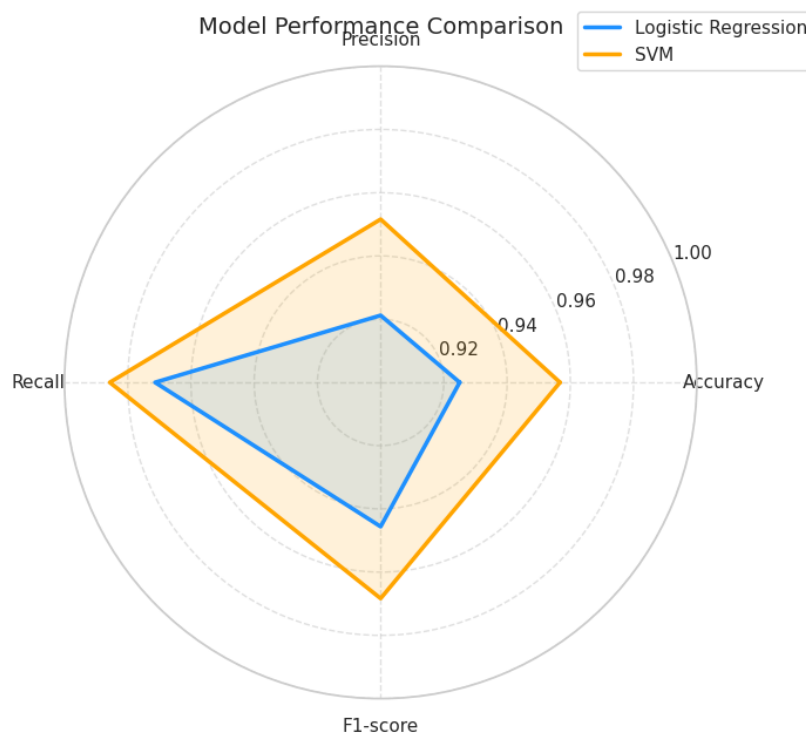
+ False Negative (FN) = 10: Có 10 trường hợp thực tế thuộc lớp 1 nhưng bị dự đoán nhầm thành lớp 0.

+ True Positive (TP) = 688: Có 688 trường hợp thực tế thuộc lớp 1 và được mô hình dự đoán chính xác.

Giống với mô hình Logistic Regression, tổng số quan sát trong tập kiểm tra là 1040 mẫu, trong đó 995 mẫu được phân loại đúng, tương ứng với độ chính xác khoảng 95.67%. Số lượng False Negative khá thấp chỉ 10 trường hợp cho thấy mô hình hầu như không bỏ sót các quan sát thuộc lớp 1. Ngoài ra, số lượng False Positive cũng ở mức thấp, phản ánh khả năng phân loại khá chính xác giữa hai lớp.

5.2.3. So sánh kết quả hai mô hình

- Các chỉ số đánh giá:



Hình 5.1. Biểu đồ radar so sánh hiệu suất của hai mô hình phân loại

Quan sát các giá trị trên biểu đồ radar có thể nhận thấy SVM đạt kết quả cao hơn Logistic Regression ở tất cả các tiêu chí.

Accuracy của SVM đạt khoảng 0.9567, trong khi Logistic Regression đạt 0.9250, cao hơn khoảng 0.0317 (tương đương 3.17%) cho thấy mô hình SVM phân loại đúng nhiều quan sát hơn trên tập dữ liệu kiểm tra.

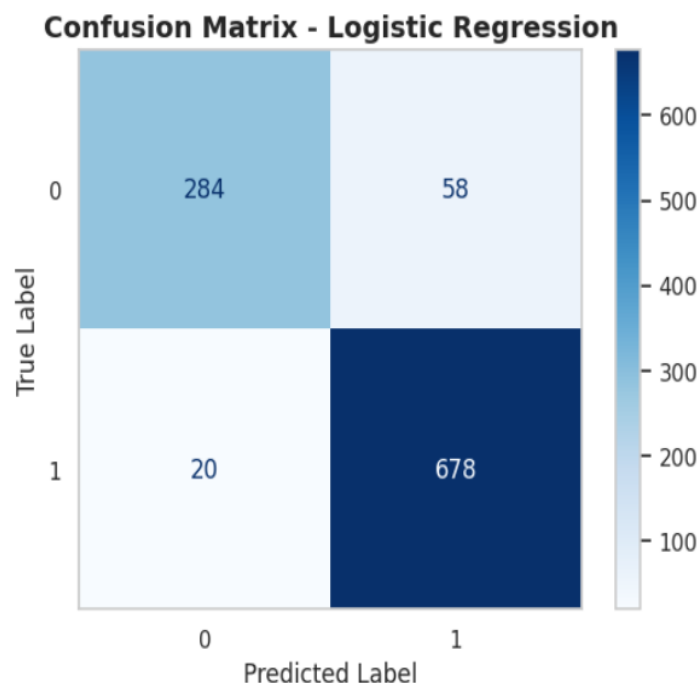
Đối với Precision, SVM đạt 0.9516, cao hơn khoảng 0.0304 (3.04%) so với 0.9212 của Logistic Regression. Các dự đoán thuộc lớp dương của SVM có độ chính xác cao hơn.

Ở chỉ số Recall, SVM đạt 0.9857, trong khi Logistic Regression đạt 0.9713, chênh lệch khoảng 0.0143 (1.43%). Kết quả này thể hiện được khả năng nhận diện của mô hình SVM là tốt hơn, nhận diện được nhiều trường hợp thuộc lớp dương hơn và ít bỏ sót dữ liệu hơn.

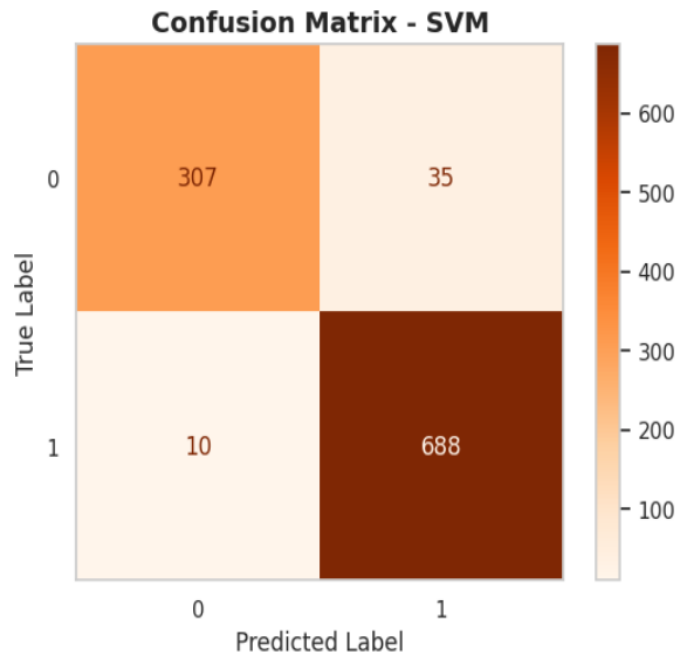
F1-score của SVM đạt 0.9683, cao hơn 0.9456 của Logistic Regression, chênh lệch khoảng 0.0227 (2.27%) phản ánh sự cân bằng tốt hơn giữa Precision và Recall của mô hình SVM.

Mặc dù cả hai mô hình đều đạt hiệu suất cao, SVM vẫn thể hiện kết quả vượt trội hơn Logistic Regression trên tất cả các chỉ số đánh giá. Do đó, trong phạm vi bộ dữ liệu và bài toán nghiên cứu này, mô hình SVM được xem là lựa chọn phù hợp hơn cho nhiệm vụ phân loại.

- *Ma trận nhầm lẫn:*



Hình 5.2. Ma trận nhầm lẫn của Logistic Regression



Hình 5.3. Ma trận nhầm lẫn của SVM

Đối với Logistic Regression, mô hình dự đoán đúng 284 trường hợp thuộc lớp 0 (True Negative) và 678 trường hợp thuộc lớp 1 (True Positive). Tuy nhiên vẫn tồn tại 58 trường hợp lớp 0 bị dự đoán nhầm thành lớp 1 (False Positive) và 20 trường hợp lớp 1 bị dự đoán nhầm thành lớp 0 (False Negative). Tổng số dự đoán đúng của mô hình là 962 trên 1040 quan sát, tương ứng với độ chính xác khoảng 92.5%.

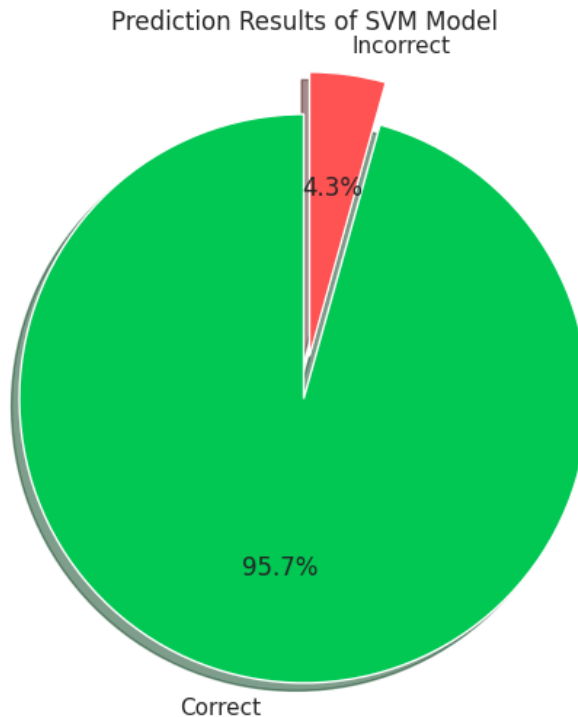
Trong khi đó, mô hình SVM đạt kết quả tốt hơn. Mô hình dự đoán đúng 307 trường hợp lớp 0 và 688 trường hợp lớp 1. Số lượng dự đoán sai giảm xuống còn 35 False Positive và 10 False Negative. Như vậy, tổng số dự đoán đúng của SVM là 995 trên 1040 quan sát, tương ứng với độ chính xác khoảng 95.67%.

Khi so sánh trực tiếp hai mô hình có thể thấy SVM giảm được 23 trường hợp False Positive (từ 58 xuống 35) và giảm 10 trường hợp False Negative (từ 20 xuống 10) so với Logistic Regression. Đồng thời số lượng dự đoán đúng ở lớp 0 tăng 23 trường hợp và ở lớp 1 tăng 10 trường hợp. Điều này cho thấy SVM không chỉ cải thiện độ chính xác tổng thể mà còn giảm đáng kể số lượng dự đoán sai ở cả hai lớp.

Dựa trên ma trận nhầm lẫn, mô hình SVM thể hiện khả năng phân loại tốt hơn Logistic Regression, đặc biệt trong việc giảm số lượng dự đoán sai và tăng số lượng

quan sát được phân loại chính xác. Vì vậy, SVM được xem là mô hình phù hợp hơn cho bài toán dự đoán trong nghiên cứu này.

5.3. Kết quả dự đoán của mô hình



Hình 5.4. Kết quả dự đoán của mô hình

Dựa trên kết quả dự đoán của mô hình SVM trên tập dữ liệu kiểm tra (test set), tổng số mẫu được sử dụng để đánh giá là 1040 mẫu. Trong đó, mô hình dự đoán chính xác 995 mẫu, chiếm khoảng 95.7% tổng số mẫu. Ngược lại, có 45 mẫu bị dự đoán sai, tương ứng với khoảng 4.3%.

Mô hình SVM đạt độ chính xác cao khi phân loại mức độ hài lòng của khách du lịch dựa trên dữ liệu đánh giá phòng khách sạn. Tỷ lệ dự đoán đúng chiếm phần lớn trong tổng số mẫu, trong khi số lượng dự đoán sai tương đối nhỏ, điều này chứng tỏ mô hình có khả năng phân loại hiệu quả giữa các trường hợp khách hàng hài lòng và không hài lòng.

5.4. Minh họa dự đoán mức độ hài lòng của khách du lịch

Để minh họa khả năng ứng dụng của mô hình trong thực tế, một bài toán giả định được xây dựng cho một khách du lịch mới với các thông tin đánh giá phòng khách sạn như sau: điểm đánh giá (Rating) là 8, số lượng đánh giá (Number of reviews) là 320,

giá phòng khoảng 80.000 BDT, loại phòng là Deluxe Double Room, loại giường 1 double bed, và địa điểm khách sạn tại Phuket.

```
# Ví dụ một khách du lịch mới
new_customer = pd.DataFrame({
    "Rating": [8],
    "Number of": [320],
    "Room Price (in BDT or any other currency)": [80000],
    "Room Type": ["Deluxe Double Room"],
    "Bed Type": ["1 double bed"],
    "Location": ["Phuket"]
})

# Dự đoán mức độ hài lòng
prediction = best_svm.predict(new_customer)
if prediction[0] == 1:
    print("Dự đoán: Khách hàng HÀI LÒNG")
else:
    print("Dự đoán: Khách hàng KHÔNG HÀI LÒNG")

prob = best_svm.predict_proba(new_customer)
print('Probability of satisfaction:', prob)
```

```
Dự đoán: Khách hàng HÀI LÒNG
Probability of satisfaction: [[0.18829466 0.81170534]]
```

Hình 5.5. Kết quả cho bài toán giả định

Các thông tin này được đưa vào mô hình Support Vector Machine (SVM) đã được huấn luyện để dự đoán mức độ hài lòng của khách hàng. Kết quả dự đoán cho thấy khách hàng này được phân loại vào nhóm hài lòng với xác suất khoảng 81.17%, trong khi xác suất thuộc nhóm không hài lòng là khoảng 18.83%.

Kết quả minh họa cho thấy mô hình không chỉ đánh giá dữ liệu đã có mà còn có khả năng dự đoán mức độ hài lòng của các khách hàng mới dựa trên các đặc trưng đầu vào. Điều này giúp mở rộng khả năng ứng dụng của mô hình trong các hệ thống hỗ trợ ra quyết định trong lĩnh vực du lịch và khách sạn.

CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Kết luận chung

Trong đề tài nghiên cứu này, bộ dữ liệu đánh giá khách sạn đã được sử dụng để xây dựng mô hình dự đoán mức độ hài lòng của khách du lịch một cách hiệu quả. Bộ dữ liệu bao gồm nhiều đặc trưng liên quan đến trải nghiệm của khách hàng như điểm đánh giá (Rating), số lượng đánh giá, giá phòng, loại phòng, loại giường và vị trí khách sạn. Các đặc trưng này phản ánh tương đối rõ những yếu tố mà khách du lịch thường quan tâm khi lựa chọn và đánh giá dịch vụ lưu trú.

Trong quá trình xây dựng mô hình, hai thuật toán học máy phổ biến là Logistic Regression và Support Vector Machine (SVM) được lựa chọn để giải quyết bài toán phân loại mức độ hài lòng của khách du lịch. Kết quả thực nghiệm cho thấy cả hai mô hình đều đạt hiệu quả phân loại tốt. Tuy nhiên, mô hình SVM đạt kết quả tốt hơn với độ chính xác khoảng 95.7%, cao hơn so với Logistic Regression đạt khoảng 92.5%. Mô hình này có khả năng phân loại và nhận diện mức độ hài lòng của khách du lịch hiệu quả hơn trên bộ dữ liệu được sử dụng.

Bên cạnh đó, phân tích ma trận nhầm lẫn cũng chứng minh số lượng dự đoán đúng chiếm phần lớn trong tổng số mẫu của tập kiểm tra, trong khi số lượng dự đoán sai chiếm tỷ lệ nhỏ. Điều này thể hiện các đặc trưng được lựa chọn trong bộ dữ liệu có khả năng phản ánh khá tốt mức độ hài lòng của khách hàng. Thử nghiệm dự đoán với một trường hợp khách du lịch giả định cũng cho thấy mô hình có thể đưa ra kết quả hợp lý về khả năng hài lòng của khách hàng dựa trên các thông tin đầu vào.

Về mặt ứng dụng thực tiễn, kết quả nghiên cứu đã chứng minh một cách hoàn hảo rằng các mô hình học máy có thể được sử dụng để dự đoán trước mức độ hài lòng của khách du lịch dựa trên dữ liệu đánh giá khách sạn. Điều này mang lại lợi ích cao khi có thể hỗ trợ các nền tảng đặt phòng trực tuyến hoặc nhà quản lý khách sạn trong việc phân tích phản hồi của khách hàng, thống kê được mức độ hài lòng và kịp thời tìm ra các giải pháp cải thiện một cách nhanh chóng. Nhờ đó, các doanh nghiệp trong lĩnh vực này có thể nâng cao mức độ hài lòng của khách hàng cũng như tăng cường khả năng cạnh tranh trên thị trường. Trong tương lai, việc mở rộng bộ dữ liệu và bổ sung thêm các đặc trưng

phản ánh sâu hơn trải nghiệm của khách hàng có thể giúp cải thiện độ chính xác của mô hình và nâng cao hiệu quả ứng dụng trong thực tế.

6.2. Hạn chế của đề tài

Mặc dù nghiên cứu đã đạt được kết quả khả quan trong việc dự đoán mức độ hài lòng của khách du lịch, đề tài vẫn còn một số hạn chế nhất định:

- + Kích thước của bộ dữ liệu còn tương đối hạn chế, do đó số lượng mẫu dùng để huấn luyện và đánh giá mô hình chưa thật sự lớn. Điều này có thể ảnh hưởng đến khả năng khái quát hóa của mô hình khi áp dụng cho các tập dữ liệu lớn hơn hoặc trong các bối cảnh thực tế khác nhau.

- + Các đặc trưng trong bộ dữ liệu vẫn chưa phản ánh đầy đủ tất cả các khía cạnh của trải nghiệm khách hàng. Dữ liệu chủ yếu tập trung vào một số thông tin cơ bản như điểm đánh giá, số lượng đánh giá, giá phòng, loại phòng, loại giường và vị trí khách sạn. Trong khi đó, mức độ hài lòng của khách du lịch còn có thể chịu ảnh hưởng bởi nhiều yếu tố khác như chất lượng dịch vụ, thái độ nhân viên, độ sạch sẽ, tiện nghi của khách sạn hoặc trải nghiệm cá nhân của từng khách hàng. Vì vậy việc dự đoán chưa thật sự tối ưu, giảm khả năng phán đoán về mức độ hài lòng của khách hàng.

- + Đề tài chỉ áp dụng hai mô hình học máy là Logistic Regression và Support Vector Machine. Đây là những phương pháp phổ biến và phù hợp cho bài toán phân loại, tuy nhiên vẫn còn nhiều thuật toán khác có thể được thử nghiệm để so sánh và cải thiện hiệu quả dự đoán.

TÀI LIỆU THAM KHẢO

- Adams, A. (2025). *Impact of online travel reviews*. Hotelagio.
<https://hotelagio.com/impact-of-online-travel-reviews/>
- Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273–297. <https://doi.org/10.1007/BF00994018>
- Machine Learning Cơ Bản. (2017). *Logistic regression*.
<https://machinelearningcoban.com/2017/01/27/logisticregression/>
- Machine Learning Cơ Bản. (2017). *Support vector machine (SVM)*.
<https://machinelearningcoban.com/2017/04/09/smv/>
- Phạm, Đ. K. (2020). *Các chỉ số đánh giá mô hình machine learning*.
<https://phamdinhkhanh.github.io/2020/08/13/ModelMetric.html>
- Phạm, Đ. K. (n.d.). *Các mô hình phân loại trong machine learning*.
https://phamdinhkhanh.github.io/deepai-book/ch_ml/index_classification.html
- Prostay. (2025). *Hotel booking statistics 2025: Market insights and trends*.
<https://www.prostay.com/blog/hotel-booking-statistics-2025-market-insights-and-trends/>
- QuarkML. (2022). *The RBF kernel in SVM: Complete guide*.
<https://www.quarkml.com/2022/10/the-rbf-kernel-in-svm-complete-guide.html>
- ScienceDirect. (n.d.). *Binary classification algorithm*.
<https://www.sciencedirect.com/topics/computer-science/binary-classification-algorithm>